Московский государственный университет имени М.В. Ломоносова
Факультет вычислительной математики и кибернетики
Кафедра суперкомпьютеров и квантовой информатики

Отчет по заданию практикума
Игра “Жизнь”

Группа: 523

Повжик Юрий Максимович
Почта: s02210514@gse.cs.msu.ru

Москва, 2025

Содержание

1 Программный код

```
#include <fstream>
#include <mpi.h>
#include <iostream>
#include <vector>
#include <set>
#include <cmath>

using namespace std;
set<int> poses;

int f(int *data, int i, int j, int n)
{
    int state = data[i * (n + 2) + j];
    int s = -state;
    for (int ii = i - 1; ii <= i + 1; ii++)
        for (int jj = j - 1; jj <= j + 1; jj++)
            s += data[ii * (n + 2) + jj];
    if (state == 0 && s == 3)
        return 1;
    if (state == 1 && (s < 2 || s > 3))
        return 0;
    return state;
}

void update_data(int n, int *data, int *temp)
{
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            temp[i * (n + 2) + j] = f(data, i, j, n);
}

void exchange_borders(int *grid, int n, int rank, int p)
{
    int row = rank / p;
    int col = rank % p;

    int up = (row == 0) ? ((p - 1) * p + col) : (rank - p);
    int down = (row == p - 1) ? col : (rank + p);
    int left = (col == 0) ? (rank + p - 1) : (rank - 1);
    int right = (col == p - 1) ? (rank - p + 1) : (rank + 1);

    MPI_Sendrecv(&grid[n], n, MPI_INT, up, 0, &grid[n * (n - 1)], n, MPI_INT,
                 down, 0, MPI_COMM_WORLD, NULL);
    MPI_Sendrecv(&grid[(n - 2) * n], n, MPI_INT, down, 1, grid, n, MPI_INT, up,
1,
                 MPI_COMM_WORLD, NULL);

    for (int i = 0; i < n; ++i)
```

```
    {
        MPI_Sendrecv(&grid[i * n + 1], 1, MPI_INT, left, 2, &grid[i * n + n - 1],
1,
                     MPI_INT, right, 2, MPI_COMM_WORLD, NULL);
        MPI_Sendrecv(&grid[i * n + n - 2], 1, MPI_INT, right, 3, &grid[i * n], 1,
                     MPI_INT, left, 3, MPI_COMM_WORLD, NULL);
    }
}

void init(int n, int *data, int nLocal, int p, int rank)
{
    for (int i = 0; i < (nLocal + 2) * (nLocal + 2); i++)
        data[i] = 0;

    int row = rank / p;
    int col = rank % p;
    int middle = (1 + n / 2) * (n + 2) + (1 + n / 2);

    for (int i = 0; i < nLocal + 2; i++)
    {
        for (int j = 0; j < nLocal + 2; j++)
        {
            int tmp = row * (n + 2) * nLocal + col * nLocal + i * (n + 2) + j;
            if (poses.find(tmp) != poses.end())
            {
                data[i * (nLocal + 2) + j] = 1;
            }
        }
    }
}

void setup_set(int n)
{
    int middle = (1 + n / 2) * (n + 2) + (1 + n / 2);
    vector<int> array = {middle + 1, middle - (n + 2),
                         middle + (n + 2) + 1, middle + (n + 2),
                         middle + (n + 2) - 1};
    for (int i = 0; i < array.size(); i++){
        poses.insert(array[i]);
    }
}

void run_life(int n, int T, int rank, int size)
{
    int p = sqrt(size) ;
    int nLocal = n / p;

    setup_set(n);
    MPI_Barrier(MPI_COMM_WORLD);
```

```
    int *dataLocal = new int[(nLocal + 2) * (nLocal + 2)];
    int *temp = new int[(nLocal + 2) * (nLocal + 2)];

    init(n, dataLocal, nLocal, p, rank);
    MPI_Barrier(MPI_COMM_WORLD);

    double time = MPI_Wtime();
    for (int t = 0; t < T; ++t)
    {
        update_data(nLocal, dataLocal, temp);
        swap(dataLocal, temp);
        exchange_borders(dataLocal, nLocal + 2, rank, p);
    }
    time = MPI_Wtime() - time;

    if (rank == 0)
    {
        int *result = new int[n * n];
        int *tmp = new int[(nLocal + 2) * (nLocal + 2)];
        for (int i = 0; i < size; i++)
        {
            int row = i / p;
            int col = i % p;
            if (i != 0)
            {
                MPI_Recv(tmp, (nLocal + 2) * (nLocal + 2), MPI_INT, i, 0,
                         MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            }
            else
                for (int j = 0; j < (nLocal + 2) * (nLocal + 2); j++)
                    tmp[j] = dataLocal[j];

            for (int j = 0; j < nLocal; j++)
            {
                for (int k = 0; k < nLocal; k++)
                {
                    result[row * n * nLocal + col * nLocal + j * n + k] =
                        tmp[(j + 1) * (nLocal + 2) + k + 1];
                }
            }
        }

        ofstream f("output.dat");
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                f << result[i * n + j];
            }
            f << endl;
```

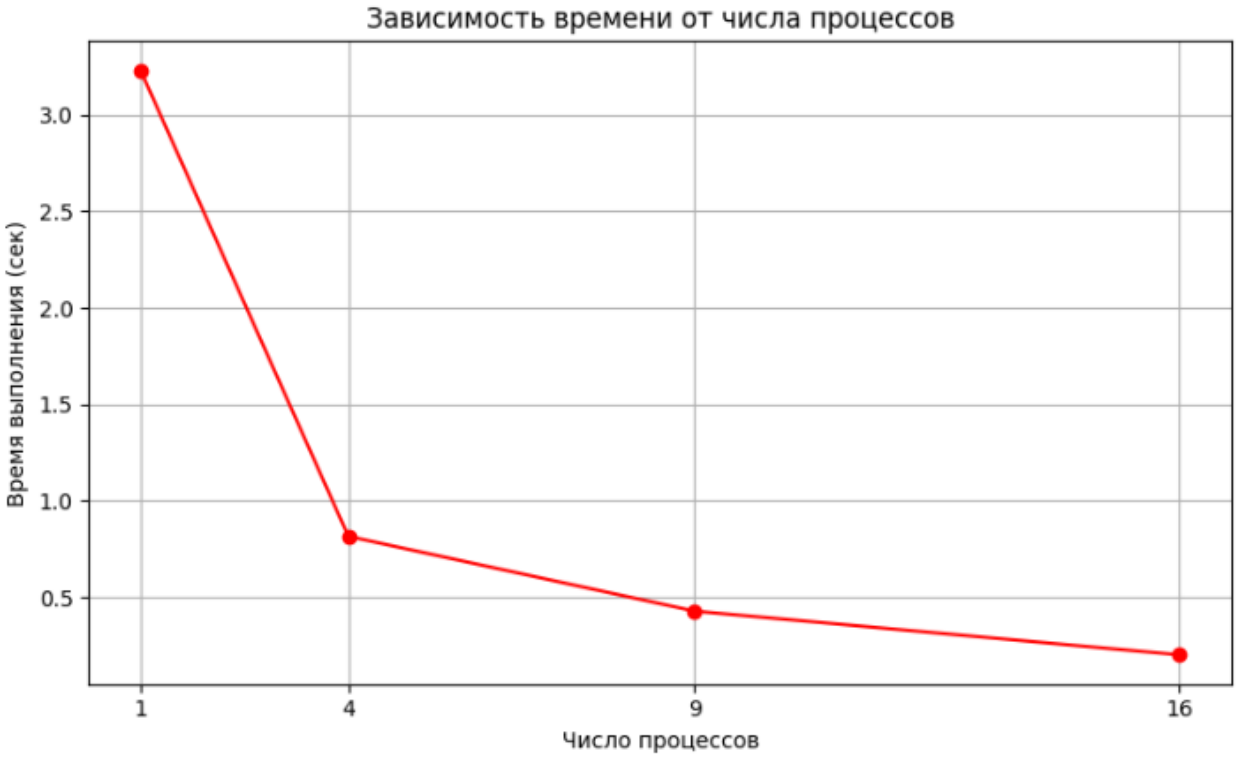
```
        }
        f.close();

        cout <<  time << endl;

        ofstream s("stat.txt");
        s << "n: " << n << endl;
        s << "T: " << T << endl;
        s << "Time: " << time << endl;
        s.close();

        delete[] tmp;
        delete[] result;
    }
    else
    {
        MPI_Send(dataLocal, (nLocal + 2) * (nLocal + 2), MPI_INT, 0, 0,
                 MPI_COMM_WORLD);
    }
    delete[] dataLocal;
    delete[] temp;
    MPI_Barrier(MPI_COMM_WORLD);
}

int main(int argc, char **argv)
{
    int size;
    int rank;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (argc != 3)
    {
        if (rank == 0)
        {
            cout << "Initial data is needed" << endl;
        }
        MPI_Barrier(MPI_COMM_WORLD);
        MPI_Finalize();
        return 0;
    }

    int n, T;

    n = atoi(argv[1]);
    T = atoi(argv[2]);

    run_life(n, T, rank, size);
```

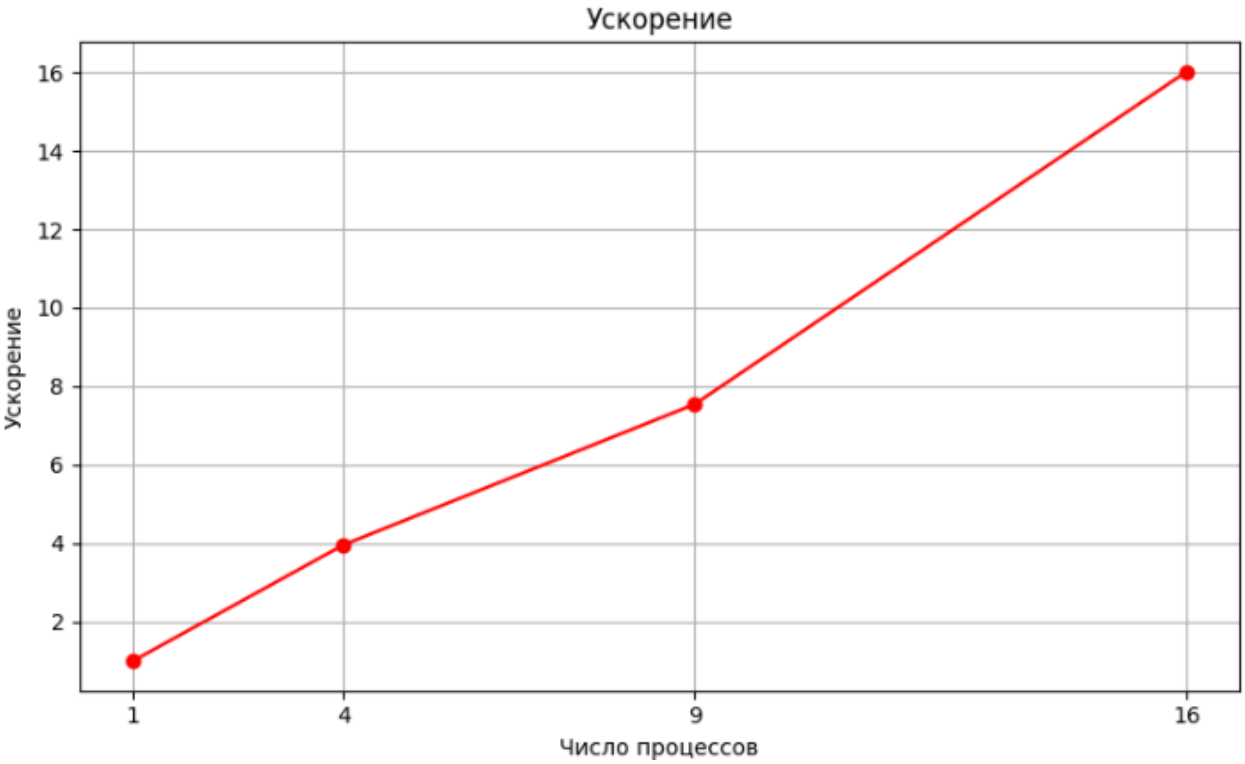
```
    MPI_Finalize();

    return 0;
}
```

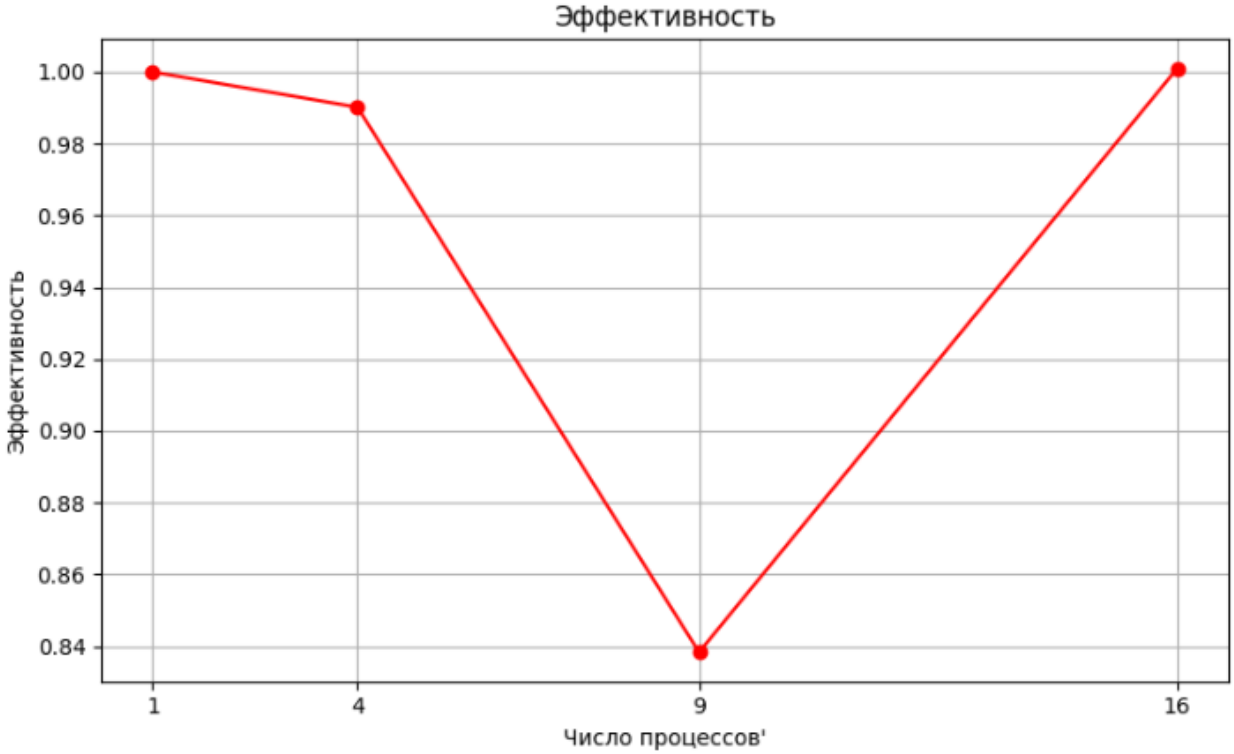
2 Графики

Входные данные:

n = 200;
T = 400;

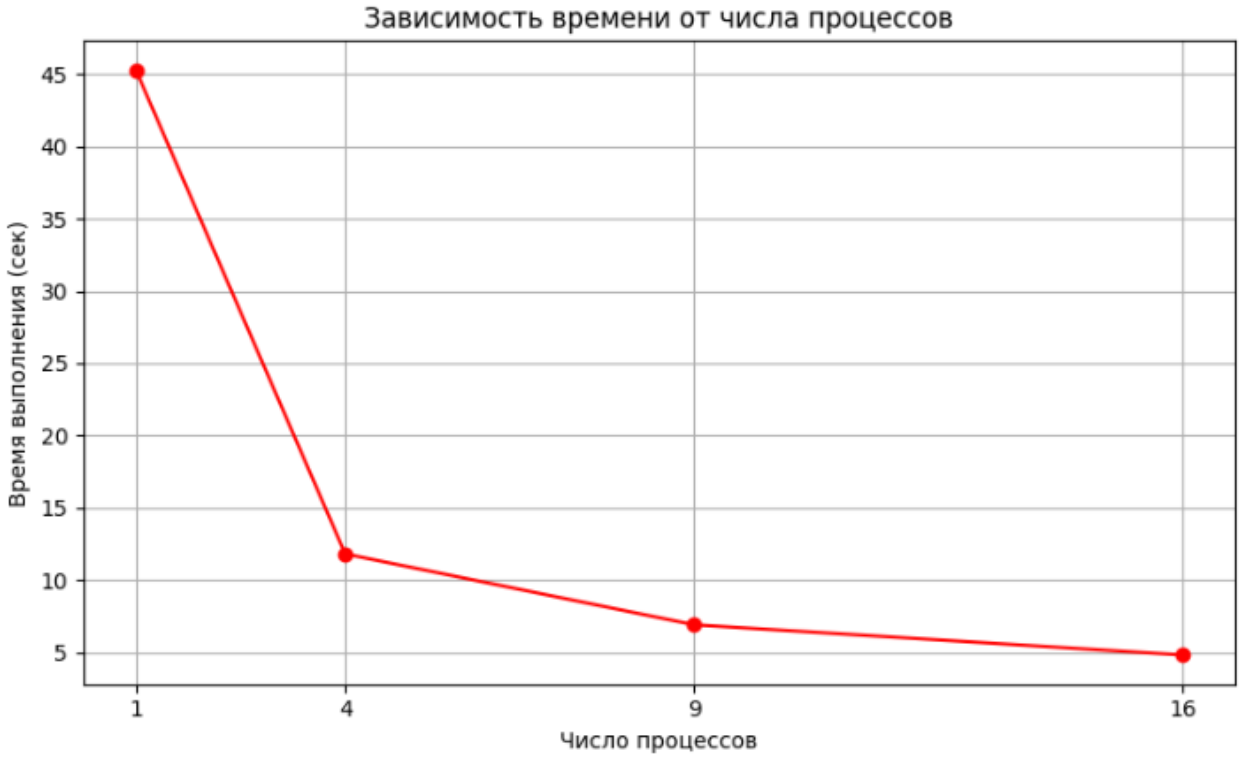


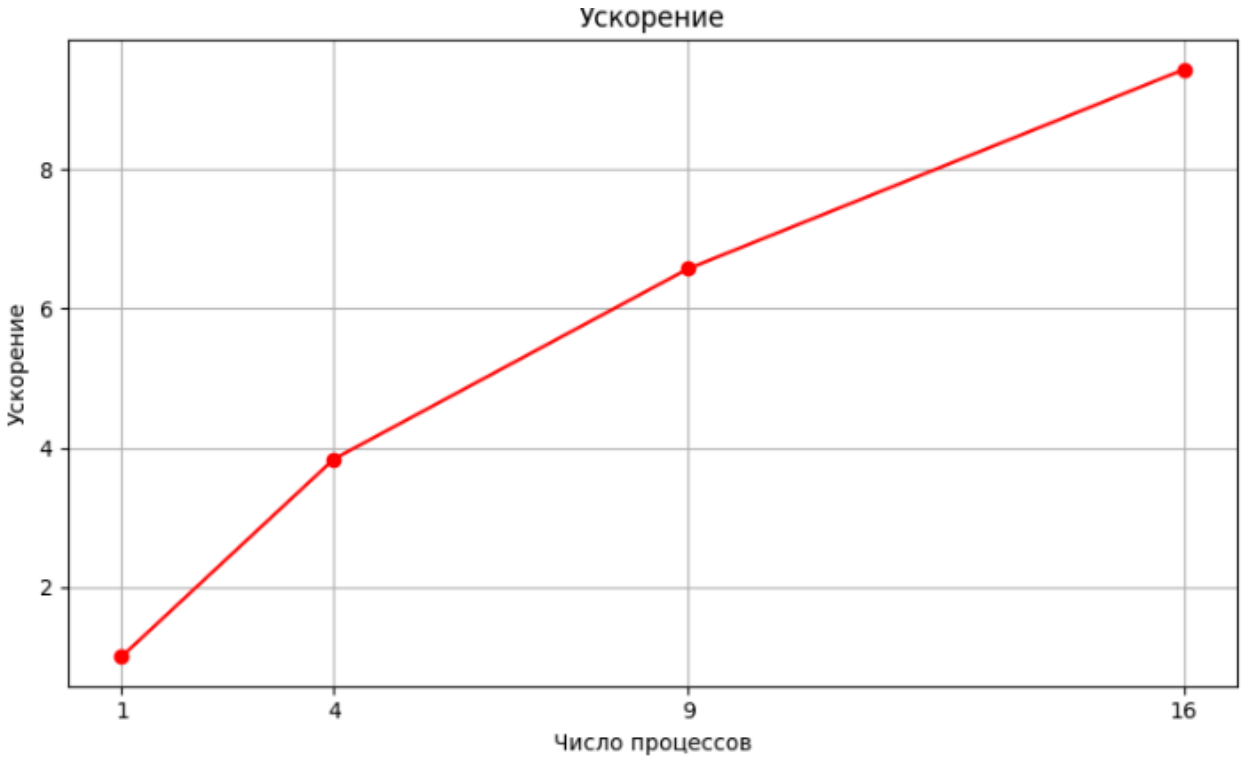

Ускорение
16
14
12
10
8
6
4
2
Ускорение
1
4
9
16
Число процессов

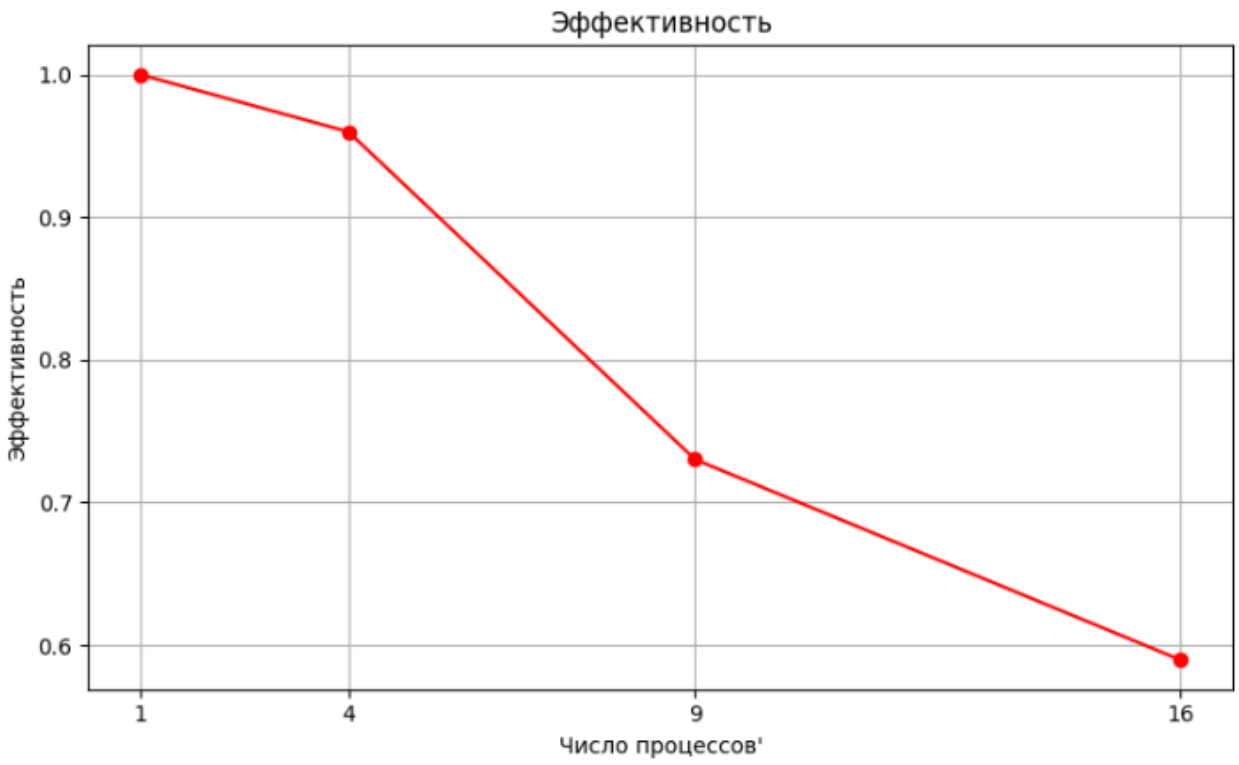

Эффективность
1.00
0.98
0.96
0.94
0.92
0.90
0.88
0.86
0.84
Эффективность
1
4
9
16
Число процессов'

Входные данные:

n = 500;
T = 2000






Эффективность
1.0
0.9
0.8
0.7
0.6
Эффективность
1
4
9
16
Число процессов'